

ORB NY — Opening Range Breakout + Retest

TradingView Pine Script v6 · 398 lines · indicators/orb-ny/ORB_NY.pine · 2026-10-01

What it does

- Builds the New York opening range (09:30 ET) on the 1, 5 and 15 minute windows.
- Watches for a candle close through the ORB High, ORB Low or 50% level.
- Confirms the entry on the retest with a Fair Value Gap, a level retest, or a strong rejection candle.
- Non-repainting: signals only on closed candles, and drawings have fixed time anchors.

Colours: ■ Entry ■ Stop loss ■ Take profit 2R ■ Breakout

To use: TradingView → Pine Editor → paste the code → Add to chart.

```
1 // This Pine Script™ code is subject to the terms of the Mozilla Public License 2.0 at https://mozilla.org/MPL/2.0/
2 // © alikamal94
3 //
4 // ORB NY — New York Opening Range Breakout + Retest
5 // 1) Builds the NY opening range on the 1, 5 and 15 minute windows (09:30 ET by default).
6 // 2) Watches for a breakout of the ORB High, ORB Low or the 50% level (candle CLOSE beyond the level).
7 // 3) Confirms the entry on the retest with a Fair Value Gap, a level (High/Low) retest, or a strong rejection candle.
8 //
9 // Non-repainting rules used in this version:
10 // - Every signal, label and line is created only on a CONFIRMED (closed) candle.
11 // - All drawings use xloc.bar_time with fixed end points, so nothing is ever moved after it is drawn.
12 // - Drawings are stored per session in var arrays; only whole old sessions are deleted (Days to keep).
13 // - One breakout label per level/direction per session, and no new setups once the daily signal limit is reached.
14 //
15 // Chart timeframe note: an ORB window can only be built when the chart timeframe fits inside it.
16 // 1m chart -> 1m, 5m and 15m ORB
17 // 5m chart -> 5m and 15m ORB
18 // 15m chart -> 15m ORB
19
20 //@version=6
21 indicator("ORB NY — Opening Range Breakout + Retest", "ORB NY", overlay = true, max_boxes_count = 500, max_lines_count = 500, max_labels_count = 500)
22
23 // ----- Inputs -----
24 const string G_SES = "New York session"
25 tz          = input.string("America/New_York", "Timezone", group = G_SES)
26 openH       = input.int(9, "Open hour", 0, 23, group = G_SES, inline = "o")
27 openM       = input.int(30, "minute", 0, 59, group = G_SES, inline = "o")
28 sigEndH     = input.int(12, "Stop new signals at hour", 0, 23, group = G_SES, inline = "s")
29 sigEndM     = input.int(0, "minute", 0, 59, group = G_SES, inline = "s")
30 lvlEndH     = input.int(16, "Levels / trades end hour", 0, 23, group = G_SES, inline = "l")
31 lvlEndM     = input.int(0, "minute", 0, 59, group = G_SES, inline = "l")
32 keepDays    = input.int(10, "Days of drawings to keep", minval = 1, maxval = 60, group = G_SES)
33
34 const string G_ORB = "Opening ranges"
35 show1       = input.bool(true, "1m ORB", group = G_ORB, inline = "1")
```

```

36 coll      = input.color(color.new(#FFB300, 0), "", group = G_ORB, inline = "1")
37 show5     = input.bool(true, "5m ORB", group = G_ORB, inline = "5")
38 col5      = input.color(color.new(#2196F3, 0), "", group = G_ORB, inline = "5")
39 show15    = input.bool(true, "15m ORB", group = G_ORB, inline = "15")
40 coll5     = input.color(color.new(#AB47BC, 0), "", group = G_ORB, inline = "15")
41 showMid    = input.bool(true, "Show 50% level", group = G_ORB)
42 showBox    = input.bool(true, "Shade range box", group = G_ORB)
43 signalTF   = input.string("Auto", "ORB used for signals", options = ["Auto", "1", "5", "15"], group = G_ORB,
44     tooltip = "Auto = same as the chart timeframe when it is 1/5/15 minutes, otherwise the 15m ORB.")
45
46 const string G_BRK = "Breakout"
47 useHL      = input.bool(true, "Trade ORB High / Low breakouts", group = G_BRK)
48 useMid     = input.bool(true, "Trade 50% level breakouts", group = G_BRK)
49 toLATR     = input.float(0.25, "Retest tolerance (× ATR)", minval = 0, step = 0.05, group = G_BRK,
50     tooltip = "How close price must come back to the broken level to count as a retest. Also the buffer used to invalidate a breakout.")
51 maxWait    = input.int(30, "Max bars to wait for the retest", minval = 1, group = G_BRK)
52 maxTrades  = input.int(1, "Max signals per session", minval = 1, group = G_BRK,
53     tooltip = "Once this many signals have fired, the session is locked: no new breakouts, labels or trades until the next NY open.")
54 showBreaks = input.bool(true, "Label breakouts / invalidations", group = G_BRK)
55
56 const string G_CNF = "Entry confirmation (any enabled one confirms)"
57 cFVG       = input.bool(true, "Fair Value Gap retest", group = G_CNF)
58 cLvl       = input.bool(true, "High / Low level retest (close holds the level)", group = G_CNF)
59 cRej       = input.bool(true, "Strong rejection candle (pin bar / engulfing)", group = G_CNF)
60 wickPct    = input.float(0.5, "Rejection: min wick as % of candle", minval = 0.1, maxval = 0.9, step = 0.05, group = G_CNF)
61 showFVG    = input.bool(true, "Draw breakout FVG", group = G_CNF)
62
63 const string G_RSK = "Risk"
64 entryMode  = input.string("Retest level", "Entry price", options = ["Retest level", "Signal candle close"], group = G_RSK,
65     tooltip = "Retest level = the broken ORB level (or the FVG edge for FVG entries). Signal candle close = close of the confirmation candle.")
66 slMode     = input.string("Retest candle", "Stop loss", options = ["Retest candle", "Opposite ORB side", "ORB 50%"], group = G_RSK,
67     tooltip = "Retest candle = beyond the high (shorts) / low (longs) of the retest candles. Opposite ORB side = ORB High for shorts, ORB Low for longs.")
68 slBufATR   = input.float(0.1, "SL buffer (× ATR)", minval = 0, step = 0.05, group = G_RSK)
69 rr         = input.float(2.0, "Take profit (R multiple)", minval = 0.1, step = 0.25, group = G_RSK)
70 tradeBars  = input.int(30, "Trade line length (bars)", minval = 5, maxval = 400, group = G_RSK)
71 showTrade  = input.bool(true, "Draw entry / SL / TP", group = G_RSK)
72 showTable  = input.bool(true, "Show info table", group = G_RSK)
73
74 // Colours
75 const color C_ENTRY = #2962FF // blue
76 const color C_SL    = #F23645 // red
77 const color C_TP     = #089981 // green
78 const color C_BRK    = #9C27B0 // purple
79
80 // ----- Time -----
81 atr      = ta.atr(14)
82 tfSec    = timeframe.in_seconds()
83 tfMs     = tfSec * 1000
84 yy       = year(time, tz)
85 mm       = month(time, tz)
86 dd       = dayofmonth(time, tz)
87 sessStart = timestamp(tz, yy, mm, dd, openH, openM)
88 sigEnd    = timestamp(tz, yy, mm, dd, sigEndH, sigEndM)

```

```

89   lvlEnd    = timestamp(tz, yy, mm, dd, lvlEndH, lvlEndM)
90   newSess   = time >= sessStart and (na(time[1]) or time[1] < sessStart)
91   closed    = barstate.isconfirmed
92
93   // ----- Drawing storage (per session) -----
94   type SessDraw
95       array<line> lines
96       array<label> labels
97       array<box> boxes
98
99   var array<SessDraw> days = array.new<SessDraw>()
100
101   regLine(line l) =>
102       if days.size() > 0
103           days.last().lines.push(l)
104       l
105
106   regLabel(label l) =>
107       if days.size() > 0
108           days.last().labels.push(l)
109       l
110
111   regBox(box b) =>
112       if days.size() > 0
113           days.last().boxes.push(b)
114       b
115
116   if newSess
117       days.push(SessDraw.new(array.new<line>(), array.new<label>(), array.new<box>()))
118       if days.size() > keepDays
119           old = days.shift()
120           for l in old.lines
121               line.delete(l)
122           for l in old.labels
123               label.delete(l)
124           for b in old.boxes
125               box.delete(b)
126
127   // ----- ORB object -----
128   type ORB
129       int mins
130       color col
131       bool show
132       bool valid
133       float hi      = na
134       float lo      = na
135       int endT      = na
136       int startTime = na
137       int readyBar  = na
138       bool building = false
139       bool ready    = false
140
141   method mid(ORB this) =>

```

```

142     (this.hi + this.lo) / 2
143
144     fits(int mins) =>
145         tfSec <= mins * 60 and (mins * 60) % tfSec == 0
146
147     var array<ORB> orbs = array.from(ORB.new(1, col1, show1, fits(1)), ORB.new(5, col5, show5, fits(5)), ORB.new(15, col15, show15, fits(15)))
148
149     for o in orbs
150         if o.valid
151             if newSess
152                 o.hi      := high
153                 o.lo      := low
154                 o.endT     := sessStart + o.mins * 60000
155                 o.startTime := time
156                 o.building := true
157                 o.ready    := false
158             else if o.building
159                 o.hi := math.max(o.hi, high)
160                 o.lo := math.min(o.lo, low)
161
162             // Lock the range only when its last candle has CLOSED, then draw it once with fixed end points
163             if o.building and closed and time_close >= o.endT
164                 o.building := false
165                 o.ready    := true
166                 o.readyBar := bar_index
167                 if o.show
168                     if showBox
169                         regBox(box.new(o.startTime, o.hi, time_close, o.lo, xloc = xloc.bar_time, border_color = color.new(o.col, 60), bgcolor = color.new(o.col, 88)))
170                         regLine(line.new(o.startTime, o.hi, lvlEnd, o.hi, xloc = xloc.bar_time, color = o.col, width = 2))
171                         regLine(line.new(o.startTime, o.lo, lvlEnd, o.lo, xloc = xloc.bar_time, color = o.col, width = 2))
172                     if showMid
173                         regLine(line.new(o.startTime, o.mid(), lvlEnd, o.mid(), xloc = xloc.bar_time, color = color.new(o.col, 30), style = line.style_dashed))
174                         regLabel(label.new(lvlEnd, o.hi, str.toString(o.mins) + "m ORB", xloc = xloc.bar_time, style = label.style_label_left,
175                                     color = color.new(color.black, 100), textcolor = o.col, size = size.small))
176
177     // ----- Signal ORB -----
178     selMins = switch signalTF
179         "1"  => 1
180         "5"  => 5
181         "15" => 15
182     => tfSec == 60 ? 1 : tfSec == 300 ? 5 : 15
183     sig = selMins == 1 ? orbs.get(0) : selMins == 5 ? orbs.get(1) : orbs.get(2)
184
185     // ----- State machine -----
186     // state: 0 = watching for breakout, 1 = breakout, waiting for retest confirmation, 2 = in trade
187     var int state = 0
188     var int dir   = 0
189     var float lvl = na
190     var string lvlName = ""
191     var int brkBar = na
192     var float fvgTop = na
193     var float fvgBot = na
194     var int fvgBar = na

```

```

195 var int    trades    = 0
196 var float  entry     = na
197 var float  sl        = na
198 var float  tp        = na
199 var int    entryBar   = na
200 // one breakout label per level/direction per session
201 var bool   lbHiUp     = false
202 var bool   lbLoDn     = false
203 var bool   lbMidUp    = false
204 var bool   lbMidDn    = false
205
206 longSig    = false
207 shortSig   = false
208 tol        = atr * tolATR
209 buf        = atr * slBufATR
210 tkr        = syminfo.ticker
211
212 if newSess
213     state  := 0
214     trades := 0
215     lbHiUp := false
216     lbLoDn := false
217     lbMidUp := false
218     lbMidDn := false
219
220 canSignal = sig.valid and sig.ready and bar_index > sig.readyBar and time >= sessStart and time < sigEnd and trades < maxTrades
221
222 // Everything below runs only on closed candles -> no intrabar signals that later disappear
223 if closed
224     // 1) Breakout detection (candle close through the level)
225     if state == 0 and canSignal
226         hi = sig.hi
227         lo = sig.lo
228         md = sig.mid()
229         int    d = 0
230         float  L = na
231         string n = ""
232         if useHL and close > hi and close[1] <= hi
233             d := 1
234             L := hi
235             n := "ORB High"
236         else if useHL and close < lo and close[1] >= lo
237             d := -1
238             L := lo
239             n := "ORB Low"
240         else if useMid and close > md and close[1] <= md and close <= hi
241             d := 1
242             L := md
243             n := "ORB 50%"
244         else if useMid and close < md and close[1] >= md and close >= lo
245             d := -1
246             L := md
247             n := "ORB 50%"

```

```

248     if d != 0
249         state := 1
250         dir := d
251         lvl := L
252         lvlName := n
253         brkBar := bar_index
254         fvgTop := na
255         fvgBot := na
256         fvgBar := na
257         isHi = n == "ORB High"
258         isLo = n == "ORB Low"
259         first = isHi ? not lbHiUp : isLo ? not lbLoDn : d == 1 ? not lbMidUp : not lbMidDn
260         if isHi
261             lbHiUp := true
262         else if isLo
263             lbLoDn := true
264         else if d == 1
265             lbMidUp := true
266         else
267             lbMidDn := true
268         if showBreaks and first
269             regLabel(label.new(time, d == 1 ? high : low, (d == 1 ? "↑ " : "↓ ") + n + " break", xloc = xloc.bar_time,
270                                 style = d == 1 ? label.style_label_down : label.style_label_up,
271                                 color = C_BRK, textcolor = color.white, size = size.tiny))
272         alert(tkr + " ORB NY: " + (d == 1 ? "bullish" : "bearish") + " breakout of " + n + " (" + str.tostring(L, format.mintick) + ") – waiting for retest",
273             alert.freq_once_per_bar_close)
274
275 // 2) Retest + confirmation
276 else if state == 1 and bar_index > brkBar
277     // Track the first FVG printed by the breakout leg
278     if na(fvgTop)
279         if dir == 1 and low > high[2]
280             fvgTop := low
281             fvgBot := high[2]
282             fvgBar := bar_index
283         else if dir == -1 and high < low[2]
284             fvgTop := low[2]
285             fvgBot := high
286             fvgBar := bar_index
287         if not na(fvgTop) and showFVG
288             regBox(box.new(time[2], fvgTop, time + 5 * tfMs, fvgBot, xloc = xloc.bar_time, border_color = na,
289                             bgcolor = color.new(dir == 1 ? C_TP : C_SL, 80), text = "FVG", text_size = size.tiny,
290                             text_color = color.new(color.gray, 20), text_halign = text.align_right))
291
292 rng = high - low
293 touchLvl = dir == 1 ? low <= lvl + tol : high >= lvl - tol
294 hold = dir == 1 ? close > lvl : close < lvl
295 dirCandle = dir == 1 ? close > open : close < open
296 wick = dir == 1 ? math.min(open, close) - low : high - math.max(open, close)
297 closePos = rng > 0 ? (dir == 1 ? close - low : high - close) / rng : 0.0
298 pinBar = rng > 0 and wick >= wickPct * rng and closePos >= 0.6
299 engulf = dir == 1 ? close > open and close[1] < open[1] and close >= open[1] and open <= close[1] :
300         close < open and close[1] > open[1] and close <= open[1] and open >= close[1]

```

```

300 touchFVG = not na(fvgTop) and bar_index > fvgBar and (dir == 1 ? low <= fvgTop and close > fvgBot : high >= fvgBot and close < fvgTop)
301
302 okFVG = cFVG and touchFVG and hold and dirCandle
303 okRej = cRej and touchLvl and hold and (pinBar or engulf)
304 okLvl = cLvl and touchLvl and hold and dirCandle
305 invalid = dir == 1 ? close < lvl - tol : close > lvl + tol
306
307 if invalid or bar_index - brkBar > maxWait or time >= sigEnd
308     state := 0
309     if showBreaks
310         regLabel(label.new(time, dir == 1 ? low : high, invalid ? "x failed" : "x no retest", xloc = xloc.bar_time,
311                             style = dir == 1 ? label.style_label_up : label.style_label_down,
312                             color = color.new(color.gray, 40), textcolor = color.white, size = size.tiny))
313     else if okFVG or okRej or okLvl
314         how = okRej ? (pinBar ? "Rejection wick" : "Engulfing") : okLvl ? "Level retest" : "FVG retest"
315         trades += 1
316         state := 2
317         entryBar := bar_index
318
319         // Entry: the broken level (FVG edge for pure FVG entries) or the signal candle close
320         fvgOnly = how == "FVG retest"
321         entry := entryMode == "Signal candle close" ? close : fvgOnly ? (dir == 1 ? fvgTop : fvgBot) : lvl
322
323         // Stop loss - LONG below, SHORT above the entry
324         retestLow = math.min(low, low[1], entry)
325         retestHigh = math.max(high, high[1], entry)
326         sl := switch slMode
327             "Opposite ORB side" => dir == 1 ? sig.lo - buf : sig.hi + buf
328             "ORB 50%" => dir == 1 ? sig.mid() - buf : sig.mid() + buf
329             => dir == 1 ? retestLow - buf : retestHigh + buf
330         risk = dir == 1 ? entry - sl : sl - entry
331         if risk <= 0
332             sl := dir == 1 ? retestLow - buf : retestHigh + buf
333             risk := dir == 1 ? entry - sl : sl - entry
334         if risk <= 0
335             risk := math.max(atr * 0.5, syminfo.mintick)
336             sl := dir == 1 ? entry - risk : entry + risk
337
338         // Take profit - LONG: entry + R*risk (above), SHORT: entry - R*risk (below)
339         tp := dir == 1 ? entry + rr * risk : entry - rr * risk
340
341         longSig := dir == 1
342         shortSig := dir == -1
343         regLabel(label.new(time, dir == 1 ? low : high, (dir == 1 ? "LONG" : "SHORT") + "\n" + lvlName + "\n" + how, xloc = xloc.bar_time,
344                             style = dir == 1 ? label.style_label_up : label.style_label_down,
345                             color = dir == 1 ? C_TP : C_SL, textcolor = color.white, size = size.small))
346         if showTrade
347             x1 = time
348             x2 = math.max(math.min(time + tradeBars * tfMs, lvlEnd), time + tfMs)
349             regLine(line.new(x1, entry, x2, entry, xloc = xloc.bar_time, color = C_ENTRY, width = 2))
350             regLine(line.new(x1, sl, x2, sl, xloc = xloc.bar_time, color = C_SL, width = 2))
351             regLine(line.new(x1, tp, x2, tp, xloc = xloc.bar_time, color = C_TP, width = 2))
352             regLabel(label.new(x2, entry, "Entry " + str.tostring(entry, format.mintick), xloc = xloc.bar_time, style = label.style_label_left,

```

```

353         color = color.new(C_ENTRY, 20), textcolor = color.white, size = size.tiny))
354     regLabel(label.new(x2, sl, "SL " + str.tostring(sl, format.mintick), xloc = xloc.bar_time, style = label.style_label_left,
355         color = color.new(C_SL, 20), textcolor = color.white, size = size.tiny))
356     regLabel(label.new(x2, tp, "TP " + str.tostring(tp) + "R " + str.tostring(tp, format.mintick), xloc = xloc.bar_time, style = label.style_label_left,
357         color = color.new(C_TP, 20), textcolor = color.white, size = size.tiny))
358     alert(tkr + " ORB NY: " + (dir == 1 ? "LONG" : "SHORT") + " @ " + str.tostring(entry, format.mintick) + " | " + lvlName + " " + how +
359         " | SL " + str.tostring(sl, format.mintick) + " | TP " + str.tostring(tp, format.mintick), alert.freq_once_per_bar_close)
360
361     // 3) Trade management (SL is checked first when both are hit in the same bar)
362     else if state == 2 and bar_index > entryBar
363         hitSL = dir == 1 ? low <= sl : high >= sl
364         hitTP = dir == 1 ? high >= tp : low <= tp
365         timeUp = time_close >= lvlEnd
366         if hitSL or hitTP or timeUp
367             state := 0
368             res = hitSL ? "SL hit" : hitTP ? "TP " + str.tostring(tp) + "R hit" : "Time exit"
369             regLabel(label.new(time, hitSL ? sl : hitTP ? tp : close, res, xloc = xloc.bar_time, style = label.style_label_left,
370                 color = color.new(hitSL ? C_SL : hitTP ? C_TP : color.gray, 30), textcolor = color.white, size = size.tiny))
371             alert(tkr + " ORB NY: " + res, alert.freq_once_per_bar_close)
372
373     alertcondition(longSig, "ORB NY Long", "ORB NY: LONG entry confirmed on retest")
374     alertcondition(shortSig, "ORB NY Short", "ORB NY: SHORT entry confirmed on retest")
375
376     // ----- Info table -----
377     var table tb = table.new(position.top_right, 4, 5, bgcolor = color.new(color.black, 20), border_width = 1, border_color = color.new(color.gray, 70))
378     if barstate.isfirst
379         table.merge_cells(tb, 0, 4, 3, 4)
380     if showTable and barstate.islast
381         txt = color.white
382         table.cell(tb, 0, 0, "ORB", text_color = txt, text_size = size.small)
383         table.cell(tb, 1, 0, "High", text_color = txt, text_size = size.small)
384         table.cell(tb, 2, 0, "50%", text_color = txt, text_size = size.small)
385         table.cell(tb, 3, 0, "Low", text_color = txt, text_size = size.small)
386         for [i, o] in orbs
387             r = i + 1
388             has = o.valid and o.ready
389             table.cell(tb, 0, r, str.tostring(o.mins) + "m" + (o.mins == selMins ? "★" : ""), text_color = o.col, text_size = size.small)
390             table.cell(tb, 1, r, has ? str.tostring(o.hi, format.mintick) : o.valid ? "..." : "n/a", text_color = txt, text_size = size.small)
391             table.cell(tb, 2, r, has ? str.tostring(o.mid(), format.mintick) : o.valid ? "..." : "n/a", text_color = txt, text_size = size.small)
392             table.cell(tb, 3, r, has ? str.tostring(o.lo, format.mintick) : o.valid ? "..." : "n/a", text_color = txt, text_size = size.small)
393     status = not sig.valid ? "Chart TF too high for " + str.tostring(selMins) + "m ORB" :
394         state == 1 ? (dir == 1 ? "↑ " : "↓ ") + lvlName + " broken – waiting for retest" :
395         state == 2 ? (dir == 1 ? "In LONG" : "In SHORT") + " | SL " + str.tostring(sl, format.mintick) + " | TP " + str.tostring(tp, format.mintick) :
396         trades >= maxTrades ? "Session locked (" + str.tostring(trades) + "/" + str.tostring(maxTrades) + " signals)" :
397         "Watching for breakout (" + str.tostring(trades) + "/" + str.tostring(maxTrades) + " today)"
398     table.cell(tb, 0, 4, status, text_color = txt, text_size = size.small)

```